



ExpressDBTree Suite v1.1

Manual

Copyright (c) 1998-1999 Developer Express Inc.
ALL RIGHTS RESERVED

Contents

Overview	4
A Self-Referenced Structure	5
Smart Record Loading	5
TdxDBTreeView	5
TdxDBTreeView Properties	6
TdxDBTreeView Methods	9
TdxDBTreeView Events	11
TdxDBTreeNode	14
TdxDBTreeNode Properties	14
TdxDBTreeNode Methods	15
TdxDBTreeNode	16
TdxDBTreeNode Properties	17
TdxDBTreeNode Methods	20
TdxDBTreeViewEdit	20
TdxDBTreeViewEdit Properties	20
TdxDBTreeViewEdit Methods	27
TdxDBTreeViewEdit Events	28
TdxTreeViewEdit	29
TdxTreeViewEdit Properties	30
TdxTreeViewEdit Methods	30
TdxTreeViewEdit	30
TdxTreeViewEdit Properties	31
TdxDBTreeViewEdit	31
TdxDBTreeViewEdit Properties	31
TdxLookupTreeView	32
TdxLookupTreeView Properties	33
TdxLookupTreeView Events	35
TdxLookupTreeView	36

TdxLookupTreeView Properties	36
TdxDBLookupTreeView	37
TdxDBLookupTreeView Properties	37
TCustomdxTreeView	38
TCustomdxTreeView Properties	38
TCustomdxTreeView Methods	39
TCustomdxTreeView Events	41
TdxTreeView	44
TdxDBTreePrintData	44
TdxDBTreePrintData Properties	44
ExpressDBTreeView Frequently Asked Questions	46
Index	52

Overview

The ExpressDBTree Suite allows you to display hierarchical data structures. A table must have a unique field and a field that refers to it.

A Self-Referenced Structure

To create a self-reference data structure you have to create a *unique* field – a *key (id)* field and a field that will be referred to a *key (id)* field – a *parent* field. The *key (id)* and *parent* fields must be of the same type.

Smart Record Loading

If your DataSet contains more than 1,000 records, you should use the Smart Record load feature.

Follow these steps to apply the smart load:

- Index (or order) the dataset by the *parent* field.
- Set *trSmartRecordLoad* in *Options*.
- Set the *RootValue* property to the *parent* value of your 'root' records. If you want to start with a particular record and to not display all records withi DBTreeView , you should assign the parent value of this particular record to *RootValue* and set *trCanDBNavigate* to False.
- Write an *OnCreateNewKeyValue* event handler. When dxDBTreeView does not load all records, the Maximum *key* field value is incorrect. You have to get the maximum *key* value to a variable when creating a form and use it when a user creates new tree nodes. Otherwise, when creating a new tree node you may get a key violation error.

Smart record loading works in the following way:

When activating a dataset, dxDBTreeView loads records which have a *parent* value equal to *RootValue* (Locate the first record, whose parent Value equals *RootValue* and then fetch and load the records whose *parent* field values are equal. That is why your dataset has to be sorted by a *parent* field). Then dxDBTreeView loads records on demands, i.e on node expansion and when dxDBTreeView has to retrieve the full structure of a loaded tree node.

TdxDBTreeView

TdxDBTreeView object is a TreeView control that can display and edit a dataset with hierarchical structures (a self-reference dataset).

Unit

dxdbtree

```
TdxDBTreeView = class(TCustomdxTreeView);
```

Description

TDBTreeView is a window that displays a hierarchical list of items related to a dataset. You can directly manipulate TreeView nodes, using drag & drop – and all changes will be reflected in the dataset. On changing the Value of a field related to DBTreeView, both label and parent properties of items change as well.

You can link TreeView to the database by specifying a data source component (TDataSource), which identifies the dataset as a value of the DBTreeView DataSource property. Set the KeyField property to a field in the dataset to be used as a key value of items. Set the ParentField property to a field in the dataset you want to use as a parent of items. Set the ListField property to a field in the dataset you want to see as individual labels of items.

TdxDBTreeView Properties

DBSelected	public / read only
DBTreeNodes	pubic / read only
DataSource	published / read/write
DisplayField	published / read/write
ImageIndexField	published / read/write
KeyField	published / read/write
ListField	published / read/write
ParentField	published / read/write
RootValue	published / read/write
SeparatedSt	published / read/write
StateIndexField	published / read/write
Options	published / read/write

DBSelected property

read only

Returns a DB item in a selected DBtree view.

property DBSelected : TdxDBTreeNode;

Description

Use DBSelected to retrieve a selected DB item in a data-aware tree-view. It is included only for compatibility with RSD DBTreeView. TdxDBTreeNode is inherited from TTreeNode:

```
MyTreeView.DBSelected= TdxDBTreeNode(MyTreeView.Selected);
```

See also:

TdxDBTreeView.DBTreeNodes, TdxDBTreeNode, TdxDBTreeNodes.

DBTreeNodes property

read only

DBTreeNodes is an indexed array of all TDBTreeNodes in a data-aware tree-view.

property DBTreeNodes : TdxDBTreeNodes;

Description

Use DBTreeNodes to directly access a particular dxDBTreeNode in a data-aware tree-view. Specify which DB item to access via Index, where the Index of 0 specifies the first item, the Index of 1 specifies the second item, etc. Each item is a TdxDBTreeNode object. Items are created automatically. It is included only for compatibility with RSD DBTreeView. TdxDBTreeNode is inherited from TTreeNode.

DataSource property

read/write

Identifies a link to the dataset where a data-aware tree view locates its data.

property DataSource: TDataSource;

Description

Set DataSource to a TDataSource object that links to a dataset where the data-aware tree-view should fetch its data. The linked dataset must have a self-reference structure.

See also:

TdxTreeView.KeyField, TdxTreeView.ParentField, TdxTreeView.ListField, TdxTreeView.DisplayField, TdxTreeView.ImageIndexField, TdxDBTreeView.StateIndexField.

DisplayField property**read/write**

Identifies a field (or fields) from which a data-aware tree-view displays data.

property DisplayField : String;

Description

Set the DisplayField property to a field name or multiple field names (separated by semicolons) whose values you would like to display for a TreeNode's Text. You must use this property if you wish to display more than one field or a field different from ListField.

See also:

TdxTreeView.ListField, TdxTreeView.SeparatedSt.

ImageIndexField property**read/write**

Specifies the field used by a data-aware tree-view control to set the ImageIndex property of a TreeNode.

property ImageIndexField: String;

Description

Set ImageIndexField to specify the field used by a data-aware tree-view control to set the ImageIndex property of a TreeNode. The dataset of the field is specified by the data source component (TDataSource). ImageIndexField must be an integer field.

See also:

TdxDBTreeView.StateIndexField

KeyField property**read/write**

Identifies the key field of a self-reference dataset.

property KeyField: string;

Description

Set the KeyField property to a key field in a self-reference dataset. The dataset is specified by the data source component (TDataSource).

See also:

Self-reference structure, TdxDBTreeView.ParentField, TdxDBTreeView.ListField.

ListField property**read/write**

Identifies a field, which displays item labels for a data-aware tree-view control.

property ListField: string;

Description

Set the ListField property to identify a field, which displays item labels for a data-aware tree-view control. The dataset of the field is specified by the data source component (TDataSource).

See also:

TdxDBTreeView.KeyField, TdxDBTreeView.ParentField, TdxDBTreeView.DisplayField.

ParentField property

read/write

Identifies the PARENT field of a self-reference dataset.

property ParentField: string;

Description

Set the ParentField property to the PARENT field of a self-reference dataset. The dataset of the field is specified by the data source component (TDataSource).

See also:

Self-reference structure, TdxDBTreeView.KeyField, TdxDBTreeView.ListField.

RootValue property

read/write

Determines the PARENT value of your 'root' records.

property RootValue : String;

Description

Set the RootValue property to the PARENT value of your 'root' records. This is a critical property when using smart record loading. When a dataset is activated, DBTreeView loads records which have a PARENT value equal to RootValue (locate the first record, whose parent Value equals RootValue and then fetch and load records when the values of their PARENT field are equal. That is why your dataset has to be sorted by the PARENT field).

See also:

The smart record loading

SeparatedSt property

read/write

Determines a string, which is used as a separator between values of different fields in a Tree Node's Text, once the DisplayField property is assigned.

property SeparatedS : string;

Description

Set SeparatedSt to determine a string which is used as a separator between values of different fields in a Tree Node's Text, once the DisplayField property is assigned. The default value is ' - '.

See also:

TdxDBTreeView.DisplayField

StateIndexField property

read/write

Specifies a field, which is used by a data-aware tree-view control to set the StateIndex property of a TreeNode.

property StateIndexField: String;

Description

Set StateIndexField to specify a field, which is used by a data-aware tree-view control to set the StateIndex property of a TreeNode. The dataset is specified by the data source component (TDataSource). StateIndexField must be an integer field.

See also:

TdxDBTreeView.ImageIndexField

Options property

read/write

Specifies various behavioral properties of a data-aware tree-view.

type

```
TdxDBTreeViewOption = (trDBCanDelete, trDBConfirmDelete, trCanDBNavigate,
trSmartRecordLoad, trSmartRecordCopy, trCheckHasChildren);
TdxDBTreeViewOptions = set of TdxDBTreeViewOption;
```

property Options : TdxDBTreeViewOptions;

Description

Set Options to include the desired properties for a data-aware tree-view. Options is a set drawn from the following values:

Values	Behavior
trDBCanDelete	Deletes the selected tree node and its structure, when a user presses the Delete key.
trDBConfirmDelete	A message box appears, asking for confirmation, when a user presses the Delete key to delete a node in the DBTreeView. trDBCanDelete must also be set.
trCanDBNavigate	On changing the DB the cursor in a linked DataSet, a selected tree node are also changed.
trSmartRecordLoad	DBTreeView uses the smart record loading feature.
trSmartRecordCopy	On drag & drop events, copies the associated data in the dataset. By default, Key, Parent and Name fields are the only items copied.
trCheckHasChildren	Checks whether a tree node includes children. By default, these options is inactive and dxDBTreeView does not check whether a node has children when using the smart record load feature. The resultant effect is that the HasChildren property of all nodes equals True (the plus symbol is shown within the tree). To check whether nodes have children (dxDBTreeView tries to find any child records) set this option. If you do not use the smart record load feature, dxDBTreeView ignores it.

TdxDBTreeView Methods

GetImageIndexField	public
GetKeyField	public
GetListField	public
GetParentField	public
GetStateIndexField	public
GotoKeyFieldValue	public
IsCustomDraw	public
RefreshItems	public

GetImageIndexField method

Returns ImageIndexField.

```
function GetImageIndexField : TField;
```

Description

Call GetImageIndexField to return ImageIndexField if a linked DataSet is Active and the ImageIndexField property has been assigned, otherwise returns Nil.

See also:

TdxDBTreeView.GetStateIndexField, TdxDBTreeView.ImageIndexField.

GetKeyField method

Returns KeyField.

```
function GetKeyField : TField;
```

Description

Call GetKeyField to return KeyField if a linked DataSet is Active and the KeyField property has been assigned, otherwise returns Nil.

See also:

TdxDBTreeView.KeyField, TdxDBTreeView.GetParentField.

GetListField method

Returns ListField.

```
function GetListField : TField;
```

Description

Call GetListField to return ListField if a linked DataSet is Active and the ListField property has been assigned, otherwise returns Nil.

See also:

TdxDBTreeView.ListField

GetParentField method

Returns ParentField.

```
function GetParentField: TField;
```

Description

Call GetParentField to return ParentField if a linked DataSet is Active and the ParentField property has been assigned, otherwise returns Nil.

See also:

TdxDBTreeView.ParentField, TdxDBTreeView.GetKeyField

GetStateIndexField method

Returns StateIndexField

```
function GetStateIndexField : TField;
```

Description

Call `GetStateIndexField` to return `StateIndexField` if a linked `DataSet` is Active and the `StateIndexField` property has been assigned, otherwise returns Nil.

See also:

TdxDBTreeView.StateIndexField, TdxDBTreeView.GetImageIndexField.

GotoKeyFieldValue method

Allows you to move to a particular `DBTreeNode` Item.

procedure GotoKeyFieldValue(Value : Variant);

Description

Call `GotoKeyFieldValue` to navigate to a `dxDBTreeNode` Item whose `KeyValue` is equal to `Value`.

See also:

TdxDBTreeView.DBSelected

IsCustomDraw method

Determines whether custom draw is used.

function IsCustomDraw : Boolean; override;

Description

Call `IsCustomDraw` to determine whether custom draw is used. It returns `True` if custom draw is used and `False` if not. Custom draw is used if you write a code within the `OnCustomDraw` event handler or if at least one `IsCustomDraw` property of a `DBTreeNode` object is set to `True`.

See also:

TdxDBTreeView.OnCustomDraw, TdxDBTreeNode.BkColor, TdxDBTreeNode.Color, TdxDBTreeNode.FontStyle, TdxDBTreeNode.FontName, TdxDBTreeNode.IsCustomDraw.

RefreshItems method

Refreshes items of a data-aware tree-view.

procedure RefreshItems;

Description

Call `RefreshItems` to refresh items of a data-aware tree-view. You may never need to call this procedure directly.

TdxDBTreeView Events

OnAddNewItem	published
OnCreateNewKeyValue	published
OnSetDisplayItemText	published

OnAddNewItem event

Occurs when `dxDBTreeView` must add a new item.

```
type
TAddNewDBTreeNodeEvent= procedure(Sender: TObject; var DBTreeNode:
TdxDBTreeNode) of Object;
```

```
property OnAddNewItem : TAddNewDBTreeNodeEvent;
```

Description

Use the OnAddNewItem event to perform particular operations when adding new items, for example to assign ImageIndex, SelectedIndex and StateIndex properties of a TreeNode item.

```
(*
In this example we use the onAddNewItem event and BkColor, Color,
FontStyle, FontName and IsCustomDraw properties of TdxDBTreeNode to
implement custom draw in TdxDBTreeView.
*)
```

```
procedure TForm1.dxDBTreeView1AddNewItem(Sender: TObject;
var DBTreeNode: TdxDBTreeNode);
begin
  case Table1.FindField('state').AsInteger of
    0:
      with DBTreeNode do begin
        FontStyle := [];
        Color := clGreen;
        BkColor := clWhite;
      end;
    1:
      with DBTreeNode do begin
        FontStyle := [fsBold];
        Color := clRed;
        BkColor := clYellow;
      end;
    2:
      with DBTreeNode do begin
        FontStyle := [fsBold];
        Color := clWhite;
        BkColor := clBlack;
      end;
    3:
      with DBTreeNode do begin
        FontStyle := [fsItalic];
        Color := clBlue;
        BkColor := clWhite;
      end;
    4:
      with DBTreeNode do begin
        FontStyle := [fsUnderLine];
        Color := clSilver;
        BkColor := clWhite;
      end;
  end;
  DBTreeNode.IsCustomDraw := True;
end;
```

OnCreateNewKeyValue event

Occurs when a data-aware tree-view has already inserted a tree node and tries to insert a new record into the dataset.

```

type
TCreateNewKeyValue = procedure(Sender: TObject; var NewKeyValue: Variant)
of Object;
property OnCreateNewKeyValue : TCreateNewKeyValue;

```

Description

Write an OnCreateNewKeyValue event handler to assign a new Key Value to the newly inserted tree node. NewKeyValue is the greatest value in the Key field plus 1 for integer and float fields. This gives a programmer the opportunity to change NewKeyValue (the key value for a new data record). It is especially useful if the key field is not integer and is not float.

Note: You must assign this event if you use SmartLoad. If dxDBTreeView does not load all records, it has an incorrect maximum value of the key field.

```

( *
In this example we use OnNewKeyValue to get KeyValue for the newly created
node from DB.
*)

```

```

procedure TForm1.dxDBTreeViewCreateNewKeyValue(Sender: TObject;
var NewKeyValue: Variant);
begin
  GetMaxIdQuery.SQL.Clear;
  GetMaxIdQuery.SQL.Add('Select max(id) From Products');
  GetMaxIdQuery.Open;
  NewKeyValue := GetMaxIdQuery.Fields[0].AsInteger + 1;
  GetMaxIdQuery.Close;
end;

```

See also:

The smart record loading

OnSetDisplayItemText event

Occurs when a data-aware tree-view has to display an item.

```

type
TSetDisplayItemText = procedure(Sender: TObject; var DisplayText: String)
of Object;
property OnSetDisplayItemText : TSetDisplayItemText;

```

Description

Write an OnSetDisplayItemText event handler to assign the tree node Text a specific value. DisplayText is a text to be displayed.

```

( *
In this example we use the OnSetDisplayItemText event to change the text
of tree nodes based on the tree node level.
Please, pay attention: Tree nodes may display any text, but you will edit
the Text that is required from the field - ListField.
*)

```

```

procedure TForm1.dxDBTreeViewSetDisplayItemText(Sender: TObject; var
DisplayText: string);
Var
  tr : TTreeNode;
begin
  // Get TreeNode
  tr :=

```

```

dxDBTreeView.DBTreeNodes.GetTreeNode(TreeQueryID.Value);
if tr <> Nil then begin
  if tr.Level mod 2 = 1 then
    DisplayText := Field1.Text
  else DisplayText := Field2.Text;
end;
end;

```

See also:

TdxDBTreeView.DisplayField, TdxDBTreeView.SeparatedSt.

TdxDBTreeNodes

Unit

dxdbtree

TdxDBTreeNodes = class (TObject);

Description

The TdxDBTreeNodes object maintains a list of items (TdxDBTreeNode objects). The Items property of a data-aware tree-view control (theTdxDBTreeView component) which is of the TdxDBTreeNodes type maintains the items in the data-aware tree-view.

Note: Items are created automatically.

TdxDBTreeNodes Properties

Items	public / read only
MaxKeyFieldValue	pubic / read only

Items property

read only

Specifies the DBdxTreeNodes index in TdxDBTreeNodes.

property Items[Index: Integer]: TDBTreeNode;

Description

The Items property contains individual items that appear in the dbtree view control. Individual nodes within TdxDBTreeView are TdxDBTreeNode objects. These items can be accessed via the Items property along with the item's index. For example, to access the first db item within the DBTreeView, you could use the following code:

```
MyDBTreeNode := DBTreeView1.DBNodes[0];
```

See also:

TdxDBTreeNode

MaxKeyFieldValue property

read only

Determines the greatest value of the dxDBTreeNode Key value.

property MaxKeyFieldValue : Variant;

Description

Use MaxKeyFieldValue to determine the greatest value of the DBTreeNode Key value.

Note: When using smart record loading, this value will be incorrect.

```
(*
In this example we use the DBTreeNode.MaxKeyFieldValue property of the
dxDBTreeView to set up the KeyField in the Table.
*)
```

```
procedure TForm1.T1AfterInsert(DataSet: TDataSet);
begin
  if varIsNull(T1.FindField('pr_id').Value) then begin
    if varIsNull(dxDBTreeView1.DBTreeNode.MaxKeyFieldValue)
    then
      T1.FindField('pr_id').AsInteger := 0
    else
      T1.FindField('pr_id').Value :=
        dxDBTreeView1.DBTreeNode.MaxKeyFieldValue + 1;
    end;
  end;
```

See also:

Smart record load

TdxDBTreeNode Methods

Count	public
GetKeyFieldValue	public
GetParentValue	public
GetDBTreeNode	public
GetTreeNode	public
IndexOf	public

Count method

Determines the number of DBTreeNode items in an object.

```
function Count : Integer;
```

Description

Call Count to determine the number of DBTreeNode items in an object.

GetKeyFieldValue method

Returns the Value of the KeyField field associated with dxDBTreeNode.

```
function GetKeyFieldValue(Value : TTreeNode) : Variant;
```

Description

Call GetKeyFieldValue to return the KeyField field Value associated with a TreeNode parameter Value.

See also:

TdxDBTreeView.GetParentValue

GetParentValue method

Returns the Value of the ParentField field associated with dxDBTreeNode.

```
function GetParentValue(Value : TTreeNode) : Variant;
```

Description

Call GetParentValue to return the Value of the ParentField field associated with a TTreeNode parameter Value.

See also:

TdxDBTreeView.GetKeyFieldValue

GetDBTreeNode method

Returns DBTreeNode items by its KeyValue.

```
function GetDBTreeNode(Value : Variant) : TDBTreeNode;
```

Description

Call GetDBTreeNode to return DBTreeNode items by its KeyValue.

GetTreeNode method

Returns TTreeNode items by its KeyValue.

```
function GetTreeNode(Value : Variant) : TTreeNode;
```

Description

Call GetTreeNode to return TTreeNode items by its KeyValue.

TdxDBTreeNode

The TdxDBTreeNode object is an individual item of the TdxDBTreeView control. dxDBTreeNode relates to a DB record.

Unit

dxdbtree

```
TdxDBTreeNode = class(TTreeNode);
```

Description

dxDBTreeNode is inherited from TTreeNode. TdxDBTreeNode contains KeyFieldValue and ParentFieldValue defined by the KeyField Value and ParentField records of a related DataSet.

```
(*
In this example we use the onAddNewItem event and BkColor, Color,
FontStyle, FontName and IsCustomDraw properties of TdxDBTreeNode to
implement custom draw in TdxDBTreeView.
*)
```

```
procedure TForm1.dxDBTreeView1AddNewItem(Sender: TObject;
  var DBTreeNode: TdxDBTreeNode);
begin
  case Table1.FindField('state').AsInteger of
    0:
      with DBTreeNode do begin
        FontStyle := [];
```



```

        Color := clGreen;
        BkColor := clWhite;
    end;
1:
    with DBTreeNode do begin
        FontStyle := [fsBold];
        Color := clRed;
        BkColor := clYellow;
    end;
2:
    with DBTreeNode do begin
        FontStyle := [fsBold];
        Color := clWhite;
        BkColor := clBlack;
    end;
3:
    with DBTreeNode do begin
        FontStyle := [fsItalic];
        Color := clBlue;
        BkColor := clWhite;
    end;
4:
    with DBTreeNode do begin
        FontStyle := [fsUnderLine];
        Color := clSilver;
        BkColor := clWhite;
    end;
end;
DBTreeNode.IsCustomDraw := True;
end;

```

TdxDBTreeNode Properties

BkColor	pubic / read/write
Color	pubic / read/write
FontStyle	pubic / read/write
FontName	pubic / read/write
IsCustomDraw	pubic / read/write
ChildLoaded	pubic / read/write
KeyFieldValue	pubic / read only
ParentFieldValue	pubic / read only

BkColor property

read/write

Specifies the background color of the tree node text.

property BkColor : TColor;

Description

Set BkColor to specify the background color of the tree node text.

Note: the IsCustomDraw property must be set to True.

See also:

TdxDBTreeNode.Color, TdxDBTreeNode.FontStyle, TdxDBTreeNode.FontName, TdxDBTreeNode.IsCustomDraw.

Color property**read/write**

Specifies the color of the tree node text.

property Color : TColor;

Description

Set Color to specify the color of the tree node text.

Note: the IsCustomDraw property must be set to True.

See also:

TdxDBTreeNode.BkColor, TdxDBTreeNode.FontStyle, TdxDBTreeNode.FontName, TdxDBTreeNode.IsCustomDraw.

FontStyle property**read/write**

Specifies the font style of the tree node text.

property FontStyle : TFontStyles;

Description

Set FontStyle to specify the font style of the tree node text.

Note: the IsCustomDraw property must be set to True.

See also:

TdxDBTreeNode.BkColor, TdxDBTreeNode.Color, TdxDBTreeNode.IsCustomDraw, TdxDBTreeNode.FontName.

FontName property**read/write**

Specifies the font name of tree node text.

property FontName : TFontName;

Description

Set FontName to specify the font name of the tree node text.

Note: the IsCustomDraw property must be set to True.

See also:

TdxDBTreeNode.BkColor, TdxDBTreeNode.Color, TdxDBTreeNode.IsCustomDraw, TdxDBTreeNode.FontStyle.

IsCustomDraw property**read/write**

Specifies whether a particular tree node draws font and color properties of TdxDBTreeView by default or by using BkColor, Color, FontStyle and FontName properties of this TdxDBTreeNode.

property IsCustomDraw : Boolean;

Description

Set `IsCustomDraw` to specify whether a particular tree node draws font and color properties of `TdxDBTreeView` by default or whether it uses `BkColor`, `Color`, `FontStyle` and `FontName` properties of this `TdxDBTreeNode`. By default, `IsCustomDraw` is set to `False`.

See also:

TdxDBTreeNode.BkColor, TdxDBTreeNode.Color, TdxDBTreeNode.FontName, TdxDBTreeNode.FontStyle.

ChildLoaded property

read/write

Determines whether `dxDBTreeNode` children are loaded.

property `ChildLoaded` : Boolean;

Description

Use `ChildLoaded` to determine whether children of a `DBTreeNode` are loaded. If you do not use the smart record loading feature, `ChildLoaded` is always `True`.

```
(*
In this example we demonstrate how to obtain the number of children of the
selected node. This code makes sense only when using SmartRecordLoad
options.
*)
```

```
procedure TForm1.Button1Click(Sender: TObject);
begin
  if dxDBTreeView1.Selected <> nil then
  begin
    with TdxDBTreeNode(dxDBTreeView1.Selected) do
    begin
      //Check does the child records are loaded.
      if ChildLoaded then
        ShowMessage(IntToStr(Count))
      else begin
        //Load the children records
        ChildLoaded := True;
        ShowMessage(IntToStr(Count));
        ChildLoaded := False;
      end;
    end;
  end;
end;
```

See also:

Smart record Loading

KeyFieldValue property

read only

Determines the Value of a KEY field associated with `dxDBTreeNode`.

property `KeyFieldValue` : Variant;

Description

Runtime and `ReadOnly`. Use `KeyFieldValue` to determine the Value of a KEY field associated with `dxDBTreeNode`.

See also

TdxDBTreeNode.ParentFieldValue, Self-reference structure.

ParentFieldValue property**read only**

Determines the Value of a PARENT field associated with DBTreeNode.

property ParentFieldValue: Variant;

Description

Use ParentFieldValue to determine the Value of a ParentFieldValue field associated with dxDBTreeNode.

See also:

TdxDBTreeNode.KeyFieldValue, Self-reference structure.

TdxDBTreeNode Methods

LoadChildren

public

LoadChildren method

Loads child tree nodes.

procedure LoadChildren(Recurse: Boolean);

Description

Call LoadChildren to load tree node children when using the smart record loading feature. Set the Recurse parameter to True to load all tree node records.

See also:

TdxDBTreeNode.ChildLoaded, Smart record loading.

TCustomdxVTreeViewEdit**Unit**

dxdbtree

TCustomdxVTreeViewEdit = **class**(TCustomControl)

Description

TCustomdxVTreeViewEdit is an abstract base type for tree-view edit components such as TdxDBTreeViewEdit and TdxDBLookUpTreeView.

Use TCustomdxVTreeViewEdit as a base class when defining a control that combines an edit box with a treeview. Do not create TCustomdxVTreeViewEdit instances. Use TdxTreeViewEdit (TdxDBTreeViewEdit) or TdxLookUpTreeView (TdxDBLookUpTreeView) to add a tree-view edit control.

TCustomdxVTreeViewEdit Properties

DividedChar

protected / read/write

TextStyle

protected / read/write

Selected	pubic / read only
CanSelectParents	published / read/write
DropDownRows	published / read/write
DropDownWidth	published / read/write
Text	published / read/write
TreeViewColor	published / read/write
TreeViewCursor	published / read/write
TreeViewFont	published / read/write
TreeViewHint	published / read/write
TreeViewImages	published / read/write
TreeViewIndent	published / read/write
TreeViewPopupMenu	published / read/write
TreeViewReadOnly	published / read/write
TreeViewShowButtons	published / read/write
TreeViewShowHint	published / read/write
TreeViewShowLines	published / read/write
TreeViewShowRoot	published / read/write
TreeViewSortType	published / read/write
TreeViewStatelImages	published / read/write

DividedChar property

read/write

Specifies a char, which separates the Text property.

property DividedChar : Char;

Description

Use DividedChar to specify a char, which separates the Text property. For instance, if DividedChar is '-', the selected item is 'San Jose' and its parents are 'CA' and 'US', the Text would be as follows: 'US-CA-San Jose'.

See also:

TCustomdxVTreeViewEdit.Text, *TCustomdxVTreeViewEdit.TextStyle*.

TextStyle property

read/write

Specifies a method of displaying TCustomAutoTreeViewEdit Text.

type

TTVTextStyle = (tvtsShort, tvtsFull);

property TextStyle : TTVTextStyle;

Description

Use TextStyle to specify a method of displaying TCustomdxVTreeViewEdit Text. The Text equals the label of the selected item if the TextStyle is tvtsShort and to the label of the selected item and all labels of its parent items separated by the DividedChar property if the TextStyle is tvtsFull.

See also:*TCustomdxVTreeViewEdit.DividedChar, TCustomdxVTreeViewEdit.Text***Selected property****read only**

Determines a selected tree node in a dropdown treeview control.

property Selected : TTreeNode;

Description

Use Selected to determine a selected tree node in a dropdown treeview control.

CanSelectParents property**read/write**

Specifies whether a user may choose a tree node with children from the dropdown treeview.

property CanSelectParents : Boolean;

Description

Use CanSelectParents to specify whether a user may choose a tree node with children from the popup tree-view. If CanSelectParents is True, then users may choose tree nodes with children. If False, they may not.

DropDownRows property**read/write**

Specifies the maximum number of tree nodes displayed at one time within the dropdown treeview.

property DropDownRows: Integer;

Description

Set DropDownRows to specify the maximum number of tree nodes displayed within the dropdown treeview.

See also:*TCustomdxVTreeViewEdit.DropDownWidth***DropDownWidth property****read/write**

Determines the width of a dropdown treeview

property DropDownWidht : Boolean;

Description

Use DropDownWidth to set the width of the dropdown treeview by default. If DropDownWidht is 0, then the width of the dropdown treeview will be the same as of the treeview editor.

Note: A user can change the DropDownWidth property at runtime by re-sizing the dropdown treeview window.

See also:*TCustomdxVTreeViewEdit.DropDownRows***Text property****read/write**

Contains text associated with a control.

property Text : String;

Description

Text equals the label of the selected item if the TextStyle is tvtsShort and the label of the selected item and all labels of its parent items separated by the DividedChar property if the TextStyle is tvtsFull.

See also:

TCustomdxVTreeViewEdit.DividedChar, TCustomdxVTreeViewEdit.TextStyle.

TreeViewColor property**read/write**

Specifies the color of a dropdown treeview control.

property TreeViewColor : TColor;

Description

Use TreeViewColor to specify the color of a dropdown treeview control.

See also:

TCustomdxVTreeViewEdit.TreeViewCursor, TCustomdxVTreeViewEdit.TreeViewFont, TCustomdxVTreeViewEdit.TreeViewHint, TCustomdxVTreeViewEdit.TreeViewImages, TCustomdxVTreeViewEdit.TreeViewIndent, TCustomdxVTreeViewEdit.TreeViewPopupMenu, TCustomdxVTreeViewEdit.TreeViewReadOnly, TCustomdxVTreeViewEdit.TreeViewShowButtons, TCustomdxVTreeViewEdit.TreeViewShowHint, TCustomdxVTreeViewEdit.TreeViewShowLines, TCustomdxVTreeViewEdit.TreeViewShowRoot, TCustomdxVTreeViewEdit.TreeViewSortType, TCustomdxVTreeViewEdit.TreeViewStateImages

TreeViewCursor property**read/write**

Specifies the cursor of a dropdown treeview control.

property TreeViewCursor : TCursor;

Description

Use TreeViewCursor to specify the cursor of a dropdown treeview control.

See also:

TCustomdxVTreeViewEdit.TreeViewColor, TCustomdxVTreeViewEdit.TreeViewFont, TCustomdxVTreeViewEdit.TreeViewHint, TCustomdxVTreeViewEdit.TreeViewImages, TCustomdxVTreeViewEdit.TreeViewIndent, TCustomdxVTreeViewEdit.TreeViewPopupMenu, TCustomdxVTreeViewEdit.TreeViewReadOnly, TCustomdxVTreeViewEdit.TreeViewShowButtons, TCustomdxVTreeViewEdit.TreeViewShowHint, TCustomdxVTreeViewEdit.TreeViewShowLines, TCustomdxVTreeViewEdit.TreeViewShowRoot, TCustomdxVTreeViewEdit.TreeViewSortType, TCustomdxVTreeViewEdit.TreeViewStateImages

TreeViewFont property**read/write**

Specifies the font of a dropdown treeview control.

property TreeViewFont : TFont;

Description

Use TreeViewFont to specify the font of a dropdown treeview control.

See also:

TCustomdxVTreeViewEdit.TreeViewColor, TCustomdxVTreeViewEdit.TreeViewCursor, TCustomdxVTreeViewEdit.TreeViewHint, TCustomdxVTreeViewEdit.TreeViewImages, TCustomdxVTreeViewEdit.TreeViewIndent, TCustomdxVTreeViewEdit.TreeViewPopupMenu,

TCustomdxVTreeViewEdit.TreeViewReadOnly, TCustomdxVTreeViewEdit.TreeViewShowButtons, TCustomdxVTreeViewEdit.TreeViewShowHint, TCustomdxVTreeViewEdit.TreeViewShowLines, TCustomdxVTreeViewEdit.TreeViewShowRoot, TCustomdxVTreeViewEdit.TreeViewSortType, TCustomdxVTreeViewEdit.TreeViewStateImages

TreeViewHint property

read/write

Specifies the hint of a dropdown treeview control.

property TreeViewHint : String;

Description

Use TreeViewHint to specify the hint of a dropdown treeview control.

See also:

TCustomdxVTreeViewEdit.TreeViewColor, TCustomdxVTreeViewEdit.TreeViewCursor, TCustomdxVTreeViewEdit.TreeViewFont,, TCustomdxVTreeViewEdit.TreeViewImages, TCustomdxVTreeViewEdit.TreeViewIndent, TCustomdxVTreeViewEdit.TreeViewPopupMenu, TCustomdxVTreeViewEdit.TreeViewReadOnly, TCustomdxVTreeViewEdit.TreeViewShowButtons, TCustomdxVTreeViewEdit.TreeViewShowHint, TCustomdxVTreeViewEdit.TreeViewShowLines, TCustomdxVTreeViewEdit.TreeViewShowRoot, TCustomdxVTreeViewEdit.TreeViewSortType, TCustomdxVTreeViewEdit.TreeViewStateImages

TreeViewImages property

read/write

Specifies images of a dropdown treeview control.

property TreeViewImages : TImageList;

Description

Use TreeViewImages to specify images of a dropdown treeview control.

See also:

TCustomdxVTreeViewEdit.TreeViewColor, TCustomdxVTreeViewEdit.TreeViewCursor, TCustomdxVTreeViewEdit.TreeViewFont, TCustomdxVTreeViewEdit.TreeViewHint, TCustomdxVTreeViewEdit.TreeViewIndent, TCustomdxVTreeViewEdit.TreeViewPopupMenu, TCustomdxVTreeViewEdit.TreeViewReadOnly, TCustomdxVTreeViewEdit.TreeViewShowButtons, TCustomdxVTreeViewEdit.TreeViewShowHint, TCustomdxVTreeViewEdit.TreeViewShowLines, TCustomdxVTreeViewEdit.TreeViewShowRoot, TCustomdxVTreeViewEdit.TreeViewSortType, TCustomdxVTreeViewEdit.TreeViewStateImages

TreeViewIndent property

read/write

Specifies the indentation of a dropdown treeview control.

property TreeViewIndent : Integer;

Description

Use TreeViewIndent to specify the indentation of a dropdown treeview control.

See also:

TCustomdxVTreeViewEdit.TreeViewColor, TCustomdxVTreeViewEdit.TreeViewCursor, TCustomdxVTreeViewEdit.TreeViewFont, TCustomdxVTreeViewEdit.TreeViewHint, TCustomdxVTreeViewEdit.TreeViewImages, TCustomdxVTreeViewEdit.TreeViewPopupMenu, TCustomdxVTreeViewEdit.TreeViewReadOnly, TCustomdxVTreeViewEdit.TreeViewShowButtons, TCustomdxVTreeViewEdit.TreeViewShowHint, TCustomdxVTreeViewEdit.TreeViewShowLines,

TCustomdxVTreeViewEdit.TreeViewShowRoot, TCustomdxVTreeViewEdit.TreeViewSortType, TCustomdxVTreeViewEdit.TreeViewStateImages

TreeViewPopupMenu property**read/write**

TreeViewPopupMenu specifies the popup menu of a dropdown treeview control.

property TreeViewPopupMenu : TPopupMenu;

Description

Use TreeViewPopupMenu to specify the popup menu of a dropdown treeview control.

See also:

TCustomdxVTreeViewEdit.TreeViewColor, TCustomdxVTreeViewEdit.TreeViewCursor, TCustomdxVTreeViewEdit.TreeViewFont, TCustomdxVTreeViewEdit.TreeViewHint, TCustomdxVTreeViewEdit.TreeViewImages, TCustomdxVTreeViewEdit.TreeViewIndent, TCustomdxVTreeViewEdit.TreeViewReadOnly, TCustomdxVTreeViewEdit.TreeViewShowButtons, TCustomdxVTreeViewEdit.TreeViewShowHint, TCustomdxVTreeViewEdit.TreeViewShowLines, TCustomdxVTreeViewEdit.TreeViewShowRoot, TCustomdxVTreeViewEdit.TreeViewSortType, TCustomdxVTreeViewEdit.TreeViewStateImages

TreeViewReadOnly property**read/write**

Specifies if ReadOnly determines whether a user can edit nodes of a dropdown treeview control.

property TreeViewReadOnly : Boolean;

Description

Use TreeViewReadOnly to specify if ReadOnly determines whether a user can edit nodea of a dropdown treeview control.

See also:

TCustomdxVTreeViewEdit.TreeViewColor, TCustomdxVTreeViewEdit.TreeViewCursor, TCustomdxVTreeViewEdit.TreeViewFont, TCustomdxVTreeViewEdit.TreeViewHint, TCustomdxVTreeViewEdit.TreeViewImages, TCustomdxVTreeViewEdit.TreeViewIndent, TCustomdxVTreeViewEdit.TreeViewPopupMenu, TCustomdxVTreeViewEdit.TreeViewShowButtons, TCustomdxVTreeViewEdit.TreeViewShowHint, TCustomdxVTreeViewEdit.TreeViewShowLines, TCustomdxVTreeViewEdit.TreeViewShowRoot, TCustomdxVTreeViewEdit.TreeViewSortType, TCustomdxVTreeViewEdit.TreeViewStateImages

TreeViewShowButtons property**read/write**

Specifies whether to display plus (+) and minus (-) buttons to the left of each parent item of a dropdown treeview control.

property TreeViewShowButtons: Boolean;

Description

If TreeViewShowButtons is True, a button will appear to the left of each parent item of a dropdown treeview control. A user can click the button to expand or collapse child items as an alternative to double-clicking the parent item.

See also:

TCustomdxVTreeViewEdit.TreeViewColor, TCustomdxVTreeViewEdit.TreeViewCursor, TCustomdxVTreeViewEdit.TreeViewFont, TCustomdxVTreeViewEdit.TreeViewHint, TCustomdxVTreeViewEdit.TreeViewImages, TCustomdxVTreeViewEdit.TreeViewIndent, TCustomdxVTreeViewEdit.TreeViewPopupMenu, TCustomdxVTreeViewEdit.TreeViewReadOnly,

TCustomdxVTreeViewEdit.TreeViewShowHint, TCustomdxVTreeViewEdit.TreeViewShowLines, TCustomdxVTreeViewEdit.TreeViewShowRoot, TCustomdxVTreeViewEdit.TreeViewSortType, TCustomdxVTreeViewEdit.TreeViewStateImages

TreeViewShowHint property

read/write

Determines the ShowHint property of a dropdown treeview control.

property TreeViewShowHint: Boolean;

Description

Use TreeViewImages to determine the ShowHint property of a dropdown treeview control.

See also:

TCustomdxVTreeViewEdit.TreeViewColor, TCustomdxVTreeViewEdit.TreeViewCursor, TCustomdxVTreeViewEdit.TreeViewFont, TCustomdxVTreeViewEdit.TreeViewHint, TCustomdxVTreeViewEdit.TreeViewImages, TCustomdxVTreeViewEdit.TreeViewIndent, TCustomdxVTreeViewEdit.TreeViewPopupMenu, TCustomdxVTreeViewEdit.TreeViewReadOnly, TCustomdxVTreeViewEdit.TreeViewShowButtons, TCustomdxVTreeViewEdit.TreeViewShowLines, TCustomdxVTreeViewEdit.TreeViewShowRoot, TCustomdxVTreeViewEdit.TreeViewSortType, TCustomdxVTreeViewEdit.TreeViewStateImages

TreeViewShowLines property

read/write

Specifies whether lines, which link child nodes of a dropdown treeview control to their corresponding parent nodes are displayed.

property TreeViewShowLines: Boolean;

Description

If TreeViewShowLines is True, the lines linking child nodes of a dropdown treeview control to their parent nodes are displayed. The nodes at the root of the hierarchy are not automatically linked. To link nodes at the root, the TreeViewShowRoot property must also be set to True.

See also:

TCustomdxVTreeViewEdit.TreeViewColor, TCustomdxVTreeViewEdit.TreeViewCursor, TCustomdxVTreeViewEdit.TreeViewFont, TCustomdxVTreeViewEdit.TreeViewHint, TCustomdxVTreeViewEdit.TreeViewImages, TCustomdxVTreeViewEdit.TreeViewIndent, TCustomdxVTreeViewEdit.TreeViewPopupMenu, TCustomdxVTreeViewEdit.TreeViewReadOnly, TCustomdxVTreeViewEdit.TreeViewShowButtons, TCustomdxVTreeViewEdit.TreeViewShowHint, TCustomdxVTreeViewEdit.TreeViewShowRoot, TCustomdxVTreeViewEdit.TreeViewSortType, TCustomdxVTreeViewEdit.TreeViewStateImages

TreeViewShowRoot property

read/write

Specifies whether lines connecting the top-level nodes of a dropdown treeview control are displayed.

property TreeViewShowRoot: Boolean;

Description

Set TreeViewShowRoot and TreeViewShowLines properties of the tree view to True to display lines in a dropdown treeview control connecting the top-level nodes to a single root.

See also:

TCustomdxVTreeViewEdit.TreeViewColor, TCustomdxVTreeViewEdit.TreeViewCursor, TCustomdxVTreeViewEdit.TreeViewFont, TCustomdxVTreeViewEdit.TreeViewHint, TCustomdxVTreeViewEdit.TreeViewImages, TCustomdxVTreeViewEdit.TreeViewIndent,

TCustomdxVTreeViewEdit.TreeViewPopupMenu, TCustomdxVTreeViewEdit.TreeViewReadOnly, TCustomdxVTreeViewEdit.TreeViewShowButtons, TCustomdxVTreeViewEdit.TreeViewShowHint, TCustomdxVTreeViewEdit.TreeViewShowLines, TCustomdxVTreeViewEdit.TreeViewSortType, TCustomdxVTreeViewEdit.TreeViewStateImages

TreeViewSortType property

read/write

Determines the sort type of a dropdown treeview control.

property TreeViewSortType : TSortType;

Description

Use TreeViewSortType to determine the sort type of a dropdown treeview control.

See also:

TCustomdxVTreeViewEdit.TreeViewColor, TCustomdxVTreeViewEdit.TreeViewCursor, TCustomdxVTreeViewEdit.TreeViewFont, TCustomdxVTreeViewEdit.TreeViewHint, TCustomdxVTreeViewEdit.TreeViewImages, TCustomdxVTreeViewEdit.TreeViewIndent, TCustomdxVTreeViewEdit.TreeViewPopupMenu, TCustomdxVTreeViewEdit.TreeViewReadOnly, TCustomdxVTreeViewEdit.TreeViewShowButtons, TCustomdxVTreeViewEdit.TreeViewShowHint, TCustomdxVTreeViewEdit.TreeViewShowLines, TCustomdxVTreeViewEdit.TreeViewShowRoot, TCustomdxVTreeViewEdit.TreeViewStateImages

TreeViewStateImages property

read/write

Determines the state images of a dropdown treeview control.

property property TreeViewStateImages : TImageList;

Description

Use TreeViewStateImages to determine the state images of a dropdown treeview control.

See also:

TCustomdxVTreeViewEdit.TreeViewColor, TCustomdxVTreeViewEdit.TreeViewCursor, TCustomdxVTreeViewEdit.TreeViewFont, TCustomdxVTreeViewEdit.TreeViewHint, TCustomdxVTreeViewEdit.TreeViewImages, TCustomdxVTreeViewEdit.TreeViewIndent, TCustomdxVTreeViewEdit.TreeViewPopupMenu, TCustomdxVTreeViewEdit.TreeViewReadOnly, TCustomdxVTreeViewEdit.TreeViewShowButtons, TCustomdxVTreeViewEdit.TreeViewShowHint, TCustomdxVTreeViewEdit.TreeViewShowLines, TCustomdxVTreeViewEdit.TreeViewShowRoot, TCustomdxVTreeViewEdit.TreeViewSortType

TCustomdxVTreeViewEdit Methods

CloseUp public

DropDown public

CloseUp method

Closes a dropdown treeview control.

procedure CloseUp(Accept: Boolean); virtual;

Description

Call `CloseUp` to programmatically close a dropdown treeview control. The `Accept` parameter determines whether to modify the field value with the value that corresponds to the selected node in a dropdown treeview control.

See also:

TCustomdxVTreeViewEdit.DropDown, *TCustomdxVTreeViewEdit.OnCloseUp*

DropDown method

Opens a dropdown treeview control so that a user can choose a tree node.

procedure DropDown; virtual;

Description

Call `DropDown` to programmatically open a dropdown treeview control. Before a treeview control is displayed, the `OnDropDown` event is generated.

See also:

TCustomdxVTreeViewEdit.CloseUp, *TCustomdxVTreeViewEdit.OnDropDown*

TCustomdxVTreeViewEdit Events

<code>OnCloseUp</code>	published
<code>OnCustomDraw</code>	published
<code>OnDropDown</code>	published
<code>OnGetSelectedIndex</code>	published
<code>OnGetImageIndex</code>	published

OnCloseUp event

Occurs immediately after a dropdown treeview control closes.

```
type
  TdxTreeViewCloseUp = procedure (Sender : TObject; Accept : Boolean) of
Object;
property OnCloseUp: TdxTreeViewCloseUp;
```

Description

Write an `OnCloseUp` event handler to respond when a dropdown treeview control is closed. When it is closed, the `Text` of a selected tree node (or the value that corresponds to the selected node for lookup controls) is assigned to the field. You may close the dropdown treeview control by yourself or by calling the `CloseUp` method.

See also:

TCustomdxVTreeViewEdit.CloseUp, *TCustomdxVTreeViewEdit.OnDropDown*.

OnCustomDraw event

Occurs when a dropdown treeview control paints a particular tree node using the custom draw services and the Win32 API requests the font and color for this particular tree node.

type

```
TTreeViewCustomDraw = procedure(Sender : TObject; TreeNode : TTreeNode;
Var AFont : TFont; Var AColor, ABkColor : TColor) of object;
property OnCustomDraw: TTreeViewCustomDraw;
```

Description

Use the OnCustomDraw event handler to implement custom draw services. The OnCustomDraw handler can set the font, color and background color of text for a particular tree node of a dropdown treeview control.

OnDropDown event

Occurs immediately before a dropdown treeview control opens.

```
property OnDropDown: TNotifyEvent;
```

Description

Write an OnDropDown event handler to perform specific actions before a dropdown treeview control is displayed to a user. A user can open it by himself or by calling the DropDown method.

See also:

TCustomdxVTreeViewEdit.DropDown, TCustomdxVTreeViewEdit.OnCloseUp.

OnGetSelectedIndex event

Occurs when a dropdown treeview assigns SelectedIndex of a selected node of a dropdown treeview.

```
type TTVEExpandedEvent = procedure(Sender: TObject; Node: TTreeNode) of
object;
property OnGetSelectedIndex: TTVEExpandedEvent;
```

Description

Write an OnGetSelectedIndex event handler to change the image index of a selected node of a dropdown treeview before it is drawn.

OnGetImageIndex event

Occurs when a treeview assigns ImageIndex of a selected node of a dropdown treeview.

```
type TTVEExpandedEvent = procedure(Sender: TObject; Node: TTreeNode) of
object;
property OnGetImageIndex: TTVEExpandedEvent;
```

Description

Write an OnGetImageIndex event handler to change the image index for a particular node of a dropdown treeview before it is drawn. For instance, the bitmap of a node can be changed to indicate node's different states.

TCustomdxTreeViewEdit

Unit

dxdbtrel

```
TCustomdxTreeViewEdit = class(TCustomdxVTreeViewEdit)
```

Description

TCustomdxTreeViewEdit is an abstract base type for treeview edit components such as TdxTreeViewEdit and TdxDBTreeViewEdit.

Use TCustomdxTreeViewEdit as a base class when defining controls that combine an edit box with a treeview. Do not create instances of TCustomdxTreeViewEdit. Use TdxTreeViewEdit or TdxDBTreeViewEdit to add a treeview edit control.

TCustomdxTreeViewEdit Properties

Items published / read only

Items property

read/write

Items are the items of a dropdown treeview control.

property Items : TTreeNode;

Description

The Items array property allows you to access a specific pointer kept in the Items property of a dropdown treeview control.

TCustomdxTreeViewEdit Methods

GetTreeNodeByText public

GetTreeNodeByText method

Returns the tree node of a dropdown treeview control by its text.

function GetTreeNodeByText(ParentTreeNode : TTreeNode; Text : String; flag : Boolean) : TTreeNode;

Description

Call GetTreeNodeByText to get the tree node of a dropdown treeview control by its text. The ParentTreeNode parameter is a tree node from which searching begins.

Set the flag parameter to True to recursively search all ParentTreeNode children. Use nil as a ParentTreeNode parameter to search the entire treeview.

TdxTreeViewEdit

Unit

dxdbtrel

TTreeViewEdit = **class**(TCustomdxTreeViewEdit);

Description

TTreeViewEdit component is a control, which combines an edit box with a treeview. Users can select an item from the treeview.

TdxTreeViewEdit Properties

Alignment

published / read/write

Alignment property

read/write

Specifies how text is aligned within the edit control.

property Alignment: TAlignment;

Description

The possible values of Alignment are as follows :

Value	Meaning
taLeftJustify	Align text on the left side of a control.
taCenter	Center text in a column.
taRightJustify	Align text on the right side of a control.

TdxDBTreeViewEdit

Unit

dxdbtree

TDBTreeViewEdit = **class**(TCustomdxTreeViewEdit);

Description

The TDBTreeViewEdit component is a data-aware control that combines an edit box with a treeview. Users can edit a field by selecting an item from the dropdown treeview.

TdxDBTreeViewEdit Properties

Field

public read only

DataField

published / read/write

DataSource

published / read/write

ReadOnly

published / read/write

Field property

read only

Specifies the TField object for a database field represented by TdxDBTreeViewEdit.

property Field: TField;

Description

Read Field to directly access the contents and properties of a database. Use Field to programmatically change the contents of a database field.

See also:

TdxDBTreeViewEdit.DataSource, TdxDBTreeViewEdit.DataField

DataField property

read/write

Specifies a field from which TdxDBTreeViewEdit displays its data.

property DataField: string;

Description

Use DataField to bind TdxDBTreeViewEdit to a field in a dataset. To fully specify a database field, both the dataset and the field within that dataset must be defined. The DataSource property of an edit box control specifies a dataset that contains DataField.

See also:

TdxDBTreeViewEdit.DataSource, TdxDBTreeViewEdit.Field.

DataSource property

read/write

DataSource links TdxDBTreeViewEdit to a dataset.

property DataSource: TDataSource;

Description

Use DataSource to link TdxDBTreeViewEdit to a dataset from where data is obtained. To fully specify a database field for an edit control, both the dataset and a field within that dataset must be defined. Use the DataField property to specify a particular field within the dataset.

See also:

TdxDBTreeViewEdit.DataField, TdxDBTreeViewEdit.Field.

ReadOnly property

read/write

Determines if a user can change the value of a field.

property ReadOnly: Boolean;

Description

Set ReadOnly to specify whether TdxDBTreeViewEdit should be used only for display purposes. If ReadOnly is True, TdxDBTreeViewEdit can only be used to display the value of a field on a current record. If False, a user can use TdxDBTreeViewEdit to edit the field's value.

TCustomdxLookupTreeView

Unit

dxdbtrel

TCustomdxLookupTreeView = **class**(TCustomdxVTreeViewEdit);

Description

TCustomdxLookupTreeView is an abstract base type for the treeview lookup components such as TdxDBLookupTreeView and TdxLookupTreeView.

Use TCustomLookUpTreeView as a base class when defining a control that combines an edit box with a data-aware treeview for the lookup purpose. Do not create instances of TCustomdxLookupTreeView. Use TdxDBLookupTreeView or TdxLookupTreeView to add a data-aware treeview lookup control.

TCustomdxLookupTreeView Properties

AssignField	published / read/write
DisplayField	published / read/write
ImageIndexField	published / read/write
ListSource	published / read/write
KeyField	published / read/write
ListField	published / read/write
Options	published / read/write
ParentField	published / read/write
RootValue	published / read/write
StateIndexField	published / read/write

AssignField property

read/write

Identifies a field in a ListSource dataset that matches the DataField value.

property AssignField: String;

Description

Use AssignField to link ListSource of the lookup TreeView to a DataSource. Though the name of a field specified as AssignField must not be the same as the name of a field specified as DataField, the two fields must have the same values. Generally, KeyField and AssignField in the lookup TreeView are the same; in this case you can leave AssignField empty.

See also:

TCustomdxLookupTreeView.KeyField

DisplayField property

read/write

Specifies the DisplayField property of a dropdown data-aware treeview control.

property DisplayField: String;

Description

Use DisplayField to specify the DisplayField property of a dropdown data-aware treeview control.

See also:

TCustomdxLookupTreeView.ListField, *TCustomdxLookupTreeView.ListSource*.

ImageIndexField property

read/write

Specifies a field used by the data-aware treeview control to set the ImageIndex property of TreeNode.

property ImageIndexField: String;

Description

Set ImageIndexField to specify a field used by the data-aware treeview control to set the ImageIndex property of TreeNode. The dataset is specified by the data source component (TDataSource). ImageIndexField must be an integer field.

See also:

TCustomdxLookupTreeView.StateIndexField

ListSource property**read/write**

Specifies DataSource of the dropdown treeview.

property ListSource: TDataSource;

Description

Use ListSource to specify DataSource of the dropdown treeview.

See also:

*TCustomdxLookupTreeView.AssignField, TCustomdxLookupTreeView.DisplayField,
TCustomdxLookupTreeView.KeyField, TCustomdxLookupTreeView.ParentField,
TCustomdxLookupTreeView.ListField*

KeyField property**read/write**

Specifies KeyField of the dropdown treeview.

property KeyField: String;

Description

Use KeyField to specify KeyField of the dropdown treeview. If AssignField is empty, the lookup tree view uses KeyField to link ListSource to the DataSource.

See also:

*TCustomdxLookupTreeView.AssignField, TCustomdxLookupTreeView.DisplayField,
TCustomdxLookupTreeView.ListSource, TCustomdxLookupTreeView.ParentField,
TCustomdxLookupTreeView.ListField*

ListField property**read/write**

Specifies ListField of the dropdown treeview.

property ListField: String;

Description

Use ListField to specify ListField of the dropdown treeview.

See also:

*TCustomdxLookupTreeView.AssignField, TCustomdxLookupTreeView.DisplayField,
TCustomdxLookupTreeView.ListSource, TCustomdxLookupTreeView.ParentField,
TCustomdxLookupTreeView.KeyField*

Options property**read/write**

Specifies Options of the dropdown treeview.

property Options: TdxDBTreeViewOptions;

Description

Use Options to specify options of the dropdown treeview.

Parent property**read/write**

Specifies Options of the dropdown treeview.

property ParentField: String;

Description

Use ParentField to specify ParentField of the dropdown treeview.

See also:

TCustomdxLookupTreeView.AssignField, TCustomdxLookupTreeView.DisplayField, TCustomdxLookupTreeView.ListSource, TCustomdxLookupTreeView.KeyField, TCustomdxLookupTreeView.ListField

RootValue property

[read/write](#)

Specifies Options of the dropdown treeview.

property RootValue: String;

Description

Use RootValue to specify RootValue of the dropdown treeview.

See also:

TdxDBTreeView.RootValue

StateIndexField property

[read/write](#)

Specifies a field used by a data-aware treeview control to set the StateIndex property of a tree node.

property StateIndexField: String;

Description

Set StateIndexField to specify a field used by a data-aware treeview control to set the StateIndex property of a tree node. The dataset is specified by the data source component (TDataSource). StateIndexField must be an integer field.

See also:

TCustomdxLookupTreeView.ImageIndexField

TCustomdxLookupTreeView Events

OnAddNewItem published

OnSetDisplayItemText published

OnAddNewItem event

Occurs when a data-aware dropdown treeview must add a new item.

type

TAddNewDBTreeNodeEvent= **procedure**(Sender: TObject; var DBTreeNode: TdxDBTreeNode) **of Object**;

property OnAddNewItem : TAddNewDBTreeNodeEvent;

Description

Write an OnAddNewItem event handler to perform specific operations when adding a new item to the data-aware dropdown treeview. You may use it to assign ImageIndex, SelectedIndex and StateIndex properties of a tree node item.

See also:

TdxDBTreeView.OnAddNewItem

OnSetDisplayItemText event

Specifies OnSetDisplayItemText of the dropdown treeview.

type

TSetDisplayItemText = **procedure**(Sender: TObject; var DisplayText: String) of Object;

property OnSetDisplayItemText : TSetDisplayItemText;

Description

Use OnSetDisplayItemText to specify OnSetDisplayItemText of the dropdown treeview.

See also:

TdxDBTreeView.OnSetDisplayItemText

TdxLookupTreeView

TLookupTreeView component is a control that combines an edit box with a data-aware treeview.

Unit

dxdbtrel

TLookupTreeView = **class**(TCustomdxLookupTreeView);

Description

TLookupTreeView allows you to provide users with a convenient data tree view for populating the Text property obtained from another dataset.

TdxLookupTreeView Properties

Alignment

published / read /write

Alignment property

read/write

Specifies how text is aligned within the edit control.

property Alignment: TAlignment;

Description

The possible values of Alignment include:

Value	Meaning
taLeftJustify	Align text on the left side of a control.
taCenter	Center text in a column.
taRightJustify	Align text on the right side of a control.

TdxDBLookupTreeView

TdxDBLookupTreeView component is a data-aware control that combines an edit box with a data-aware tree-view.

Unit

dxdbtrel

TdxDBLookupTreeView = **class** (TCustomdxLookupTreeView)

Description

Use TdxDBLookupTreeView to provide users with a convenient data-aware dropdown treeview for editing fields that obtain data from other datasets.

TdxDBLookupTreeView Properties

Field	public / read only
DataField	published / read/write
DataSource	published / read/write
ReadOnly	published / read/write

Field property

read only

Specifies a TField object for a database field represented by TdxDBLookupTreeView.

property Field: TField;

Description

Read Field to directly access the contents and properties of the database. Use Field to programmatically change the contents of the database field.

See also:

TdxDBLookupTreeView.DataSource, TdxDBLookupTreeView.DataField.

DataField property

read/write

Specifies a field from which TdxDBTreeViewEdit displays data.

property DataField: string;

Description

Use DataField to bind TdxDBTreeViewEdit to a field in the dataset. To fully specify a database field, both the dataset and the field within this dataset must be defined. The DataSource property of the edit box control specifies a dataset containing DataField.

See also:

TdxDBLookupTreeView.DataSource, TdxDBLookupTreeView.Field.

DataSource property

read/write

Links TdxDBLookupTreeView to a dataset, which contains the field it represents.

property DataSource: TDataSource;

Description

Use DataSource to link TdxDBLookupTreeView to a dataset in which the data are found. To fully specify a database field for the edit control, both the dataset and the field within this dataset must be defined. Use the DataField property to specify a particular field within the dataset.

See also:

TdxDBLookupTreeView.DataField, TdxDBLookupTreeView.Field.

ReadOnly property**read/write**

Determines whether a user can change the value of a field.

property ReadOnly: Boolean;

Description

Set ReadOnly to specify whether TdxDBLookupTreeView is used only for display purposes. If ReadOnly is True, TdxDBLookupTreeView can be used only for display purposes. If False, a user can use TdxDBLookupTreeView to edit a field's value.

TCustomdxTreeView

Unit

dxtree

TCustomdxTreeView = **class**(TCustomTreeView);

Description

TCustomdxTreeView is an abstract base type for treeview components such as TdxTreeView and TdxDBTreeView.

Use TCustomdxTreeView as a base class when defining components that present information in a hierarchical structure with nodes that can expand or collapse. Drag & drop operations and new methods and features are implemented within TCustomdxTreeView.

IMPORTANT: Do not create instances of TCustomdxTreeView. To add a tree view control with extended features (such as drag & drop, automatic node expansion during drag & drop operations, custom draw, etc.) instead use TdxTreeView. Use the TdxDBTreeView control to bind the TreeView with database information.

TCustomdxTreeView Properties

Options	protected / read /write
SelectedIndex	protected / read/write
DragSourceTreeNode	published / read only
ShowNodeHint	published / read/write

Options property**read/write**

Specifies various behavioral properties of dxTreeView.

type

TdxTreeViewOption = (trCanDelete, trConfirmDelete);

```
TdxTreeViewOptions = set of TdxTreeViewOption;
property Options : TdxTreeViewOptions;
```

Description

Set Options to include the desired properties for dxTreeView. Options is a set drawn from the following values:

Values	Behaviour
TrCanDelete	Deletes the selected tree node and its structure, when a user presses Delete.
TrConfirmDelete	A message box appears, asking for confirmation, when a user presses Delete to remove a node in dxTreeView. trCanDelete must also be set.

SelectedIndex property**read/write**

Specifies the index within the image list to be used when an item is selected.

```
property SelectedIndex : Integer;
```

Description

Use SelectedIndex to specify the index within the image list to be used when an item is selected.

DragSourceTreeNode property**read only**

Determines the drag source tree node during drag & drop operations.

```
property DragSourceTreeNode: TTreeNode;
```

Description

Use DragSourceTreeNode to determine the drag source tree node during drag & drop operations. You may use it in drag & drop events.

ShowNodeHint property**read/write**

Determines if dxTreeView should display a tree node hint when the mouse pointer rests momentarily on a tree node.

```
property ShowNodeHint : Boolean;
```

Description

Set ShowNodeHint to False to prevent tree node hints from displaying within all Express TreeViews.

Note: Tree node hint behavior depends on your commctrl.dll version.

TCustomdxTreeView Methods

DoCustomDraw	protected
DoDragDropTreeNode	protected
DoDragDropIsCopy	protected
IsCustomDraw	public
MoveTreeNodeStructure	public

DoCustomDraw method

DoCustomDraw is a protected implementation method for the control's OnCustomDraw event.

```
procedure DoCustomDraw(TreeNode : TTreeNode; Var AFont : TFont; Var AColor, ABkColor : TColor); virtual;
```

Description

The DoCustomDraw method does nothing except for calling any event handler attached to the OnCustomDraw event.

When creating a component that descends from TCustomdxTreeView, override DoCustomDraw to provide other responses in addition to the inherited event handler call.

Note: This method is overridden in the dxDBTreeView controls.

See also:

TCustomdxTreeView.IsCustomDraw, TCustomdxTreeView.OnCustomDraw.

DoDragDropTreeNode method

DoDragDropTreeNode is a protected implementation method for the control's OnDragDropTreeNode event.

```
procedure DoDragDropTreeNode(Destination, Source : TTreeNode; Var Accept : Boolean); virtual;
```

Description

The DoDragDropTreeNode method does nothing except for calling any event handler attached to the OnDragDropTreeNode event.

When creating a component that descends from TCustomdxTreeView, override DoDragDropTreeNode to provide other responses in addition to the inherited event handler call.

See also:

TCustomdxTreeView.OnDragDropTreeNode

DoDragDropsCopy method

DoDragDropsCopy is a protected implementation method for the control's OnDragDropsCopy event.

```
procedure DoDragDropIsCopy(Destination, Source : TTreeNode; var IsCopy : Boolean); virtual;
```

Description

The DoDragDropsCopy method does nothing except for calling any event handler attached to the OnDragDropsCopy event.

When creating a component that descends from TCustomdxTreeView, override DoDragDropsCopy to provide other responses in addition to the inherited event handler call.

See also:

TCustomdxTreeView.OnDragDropsCopy

IsCustomDraw method

Determines whether custom draw is used.

```
function IsCustomDraw : Boolean; virtual;
```


Description

Call IsCustomDraw to determine whether custom draw is used. It returns True if custom draw is used and False if not. Custom draw is used if you write a code within the OnCustomDraw event handler.

Note: This method is overridden in the dxDBTreeView controls.

See also:

TCustomdxTreeView.DoCustomDraw, TCustomdxTreeView.OnCustomDraw.

MoveTreeNodeStructure method

Moves or copies the structure of a tree node.

function MoveTreeNodeStructure (Source, Destination : TTreeNode; IsCopy : Boolean) : TTreeNode;

Description

The Source parameter determines which tree node to move or copy. The Destination parameter determines where to move or copy the tree node. The Destination and Source tree node may have different TreeViews. The IsCopy parameter determines whether the structure is moved or copied. If IsCopy is True, then the structure is copied. If False, the structure is moved. As a result you have the root of the newly created tree node structure.

TCustomdxTreeView Events

OnCustomDraw	published
OnDragDropIsCopy	published
OnDragDropTreeNode	published
OnEndDragTreeNode	published

OnCustomDraw event

Occurs when a control paints a particular tree node by using custom draw services and the Win32 API requests the font and color for this particular tree node.

type

TTreeViewCustomDraw = procedure(Sender : TObject; TreeNode : TTreeNode; Var AFont : TFont; Var AColor, ABkColor : TColor) of object;

property OnCustomDraw : TTreeViewCustomDraw;

Description

Use the OnCustomDraw event handler to implement custom draw services. The OnCustomDraw handler can set the font, color, and background color of the text for a particular TreeNode.

```
(*
This example demonstrates how to use the OnCustomDraw event.
Nodes that have KeyValue less than 150 draw with the bold font style and
blue font color, all other nodes draw with the Red color style on the
yellow background.
*)
procedure TForm1.DBTreeView1CustomDraw(Sender: TObject;
  TreeNode: TTreeNode; AFont: TFont; var AColor, ABKColor: TColor);
Var
```

```

    KeyValue : Integer;
begin
    if VarIsNull(TdxDBTreeNode(TreeNode).KeyFieldValue) then exit;
    //Get key Value of the particular node.
    KeyValue := TdxDBTreeNode(TreeNode).KeyFieldValue;
    //Change the Font Style, Color and background
    //color of the node based on it's KeyValue;
    if(KeyValue < 150) then begin
        AFont.Style := [fsBold];
        AColor := clBlue;
    end else begin
        ABKColor := clYellow;
        AColor := clRed;
    end;
end;
end;

```

See also:

TCustomdxTreeView.IsCustomDraw

OnDragDropsCopy event

Occurs when a user drags an object over a control or when a user drops an object being dragged.

```

type
TDragDropIsCopy = procedure(Destination, Source : TTreeNode;
    var IsCopy : Boolean) of object;
property OnDragDropIsCopy : TdragDropIsCopy;

```

Description

Use the OnDragDropsCopy event to change the dxTreeView drop operation behavior. Within the OnDragDropsCopy event handler, set IsCopy to True to copy the Source tree node and to False to move the Source tree node.

The Source is the tree node being dragged, the Destination is the selected drop target tree node.

```

(*)
In the following example we copy the tree node structure during drag &
drop even if a user does not press the CTRL key.
*)
procedure TForm1.DBTreeView2DragDropIsCopy(Destination, Source: TTreeNode;
    var IsCopy: Boolean);
begin
    IsCopy := True;
end;

```

OnDragDropTreeNode event

Occurs when a user drags an object over a control or when a user drops an object being dragged.

```

type
TDragDropTreeNode = procedure(Destination, Source : TTreeNode; Var Accept
    : Boolean) of object;
property OnDragDropTreeNode : TDragDropTreeNode;

```

Description

Use an OnDragDropTreeNode event to signal that dxTreeView has to copy. Set the Accept parameter to False within the OnDragDropTreeNode event handler to reject the dragged object. Set Accept to True to allow a user to drop the dragged object onto the control.

The Source is the tree node being dragged, the Destination is the selected drop target tree node.

```
(*
In the following example we disable drag & drop for root nodes.
*)
procedure TForm1.DBTreeView1DragDropTreeNode(Destination,
    Source: TTreeNode; var Accept: Boolean);
begin
    Accept := Not (Source.Parent = Nil);
end;
```

See also:

TCustomdxTreeView.DoDragDropTreeNode, TCustomdxTreeView.OnDragDropsCopy, TCustomdxTreeView.OnEndDragTreeNode.

OnEndDragTreeNode event

Occurs when the drag & drop operation is successfully finished before treenode(s) (records for data-aware TreeView) are moved (added).

```
TEndDragTreeNode = procedure(Destination, Source : TTreeNode;
    Var AttachMode : TNodeAttachMode) of object;
property OnEndDragTreeNode : TEndDragTreeNode;
```

Description

Write an OnEndDragTreeNode event handler to change the default behavior of dxTreeView on the Drop operation. The Destination and Source variants are destination and source tree nodes. The AttachMode variant is a set of possible ways in which the Source can be attached to the Destination. The default value of AttachMode is naAddChild.

Set AttachMode to:

naAdd	Adds the source to the end of the Destination list;
naAddFirst	Adds the source at the beginning of the Destination list;
naAddChild	Adds the source as a child of the destination at the end of the child list;
naAddChildFirst	Adds the source as a child of the destination at the beginning of the Destination child list;
naInsert	Inserts the source after the destination node.

```
(*
In this example we use the OnEndDragTreeNode event to change the drag&drop
behavior of TdxTreeView in the following way:
if the Source and Destination tree nodes were at the same level, then the
source tree node is inserted before the destination (the default behavior
-is the source tree node becomes the last child tree node of the
destination)
*)
procedure TForm2.DBTreeView1EndDragTreeNode(Destination,      Source:
TTreeNode;
    var AttachMode: TNodeAttachMode);
begin
    // naAdd, naAddFirst, naAddChild, naAddChildFirst, naInsert
    if Destination.Level = Source.Level then
        AttachMode := naInsert;
    if Destination.Level = Source.Level - 1 then
        AttachMode:=naAddChild;
```

```
end;
```

See also:

TCustomdxTreeView.OnDragDropTreeNode, TCustomdxTreeView.OnDragDropTreeNode

TdxTreeView

Unit

dxtree

```
TdxTreeView = class(TCustomdxTreeView);
```

Description

TdxTreeView is a treeview component with full drag & drop and custom draw support. Use TdxDBTreeView to bind your DB with a treeview control.

TdxDBTreePrintData

The TdxDBTreePrintData component is a dataset allowing easy reports printing using self-referenced datasets.

Unit

dxtrprds

```
TdxDBTreePrintData= class(TdxMemData);
```

Description

Use TdxDBTreePrintData to create a report using self-referenced datasets.

TdxDBTreePrintData is directly inherited from TdxMemData. It retrieves Data from a self-referenced DataSet (linked with the DataSource property) and creates records sorted in a specific order in memory. Additionally it creates the following fields:

Field Name	Meaning
dx\$haschildren	Determines whether a record has children. If the 'dx\$haschildren' field is True, then it has children. If False, it has none.
dx\$Level	The level of a record. The first level (level of the 'root' records) is 0.
dx\$recno	The number of a record. The first number is 1.

TdxDBTreePrintData Properties

MaxLevelCount	public / read only
DataSource	published / read/write
KeyField	published / read/write
LevelCount	published / read/write
ParentField	published / read/write
RootValue	published / read/write

MaxLevelCount property**read only**

Determines the maximum level number in TdxDBTreePrintData.

property MaxLevelCount : Integer;

Description

Use MaxLevelCount to determine the maximum level number in TdxDBTreePrintData.

DataSource property**read/write**

Identifies the link to a self-referenced dataset from where TdxDBTreePrintData fetches its data.

property DataSource : TDataSource;

Description

Set DataSource to the TDataSource object that links to a self-referenced dataset from where TdxDBTreePrintData must fetch its data.

See also:

TdxDBTreePrintData.KeyField, TdxDBTreePrintData.ParentField.

KeyField property**read/write**

Identifies the KEY field of a self-referenced dataset.

property KeyField: string;

Description

Set the KeyField property to the KEY field of the self-referenced dataset. The dataset is specified by the data source component (TDataSource).

See also:

TdxDBTreePrintData.DataSource, TdxDBTreePrintData.ParentField

RootValue property**read/write**

Determines the KEY field Value.

property RootValue : String;

Description

Use RootValue to specify the 'root' record. You can make any record of a self-referenced DataSet the root record of TdxDBTreePrintData. If RootValue is different from NULL, then TdxDBTreePrintData will load the record and all its children if its KEY value equals RootValue (the children are loaded recursively).

See also:

TdxDBTreePrintData.LevelCount

LevelCount property**read/write**

Specifies the level's number TdxDBTreePrintData can load.

property LevelCount : Integer;

Description

Set LevelCount to specify the level's number TdxDBTreePrintData can load. If LevelCount equals 0, then TdxDBTreePrintData recursively loads 'root' records and all their children. If LevelCount equals 1, then TdxDBTreePrintData loads only 'root' records. If LevelCount equals 2, then TdxDBTreePrintData loads 'root' records and their 'nearest' children, etc.

ParentField property

read/write

Identifies the PARENT field of a self-referenced dataset.

property ParentField: string;

Description

Set the ParentField property to the PARENT field of a self-referenced dataset. The dataset is specified by the data source component (TDataSource).

See also:

TdxDBTreePrintData.KeyField, TdxDBTreePrintData.DataSource

ExpressDBTreeView Frequently Asked Questions

Question 1:

Why once having assigned the ImageField & Images property, each time I move the cursor to any node, the image is replaced with the first image in the image list?

Answer:

This behavior occurs due to the SelectedIndex property of the DBTreeView is 0. If you want the selected index to be the same as the image index, then you have to write the following code on the onChange event handler of the DBTreeView:

```
procedure TForm1.DBTreeView1Change(Sender: TObject; Node: TTreeNode);
begin
    Node.SelectedIndex := Node.ImageIndex;
end;
//-----
void __fastcall TForm1::DBTreeView1Change(TObject *Sender, TTreeNode
*Node)
{
    Node->SelectedIndex = Node->ImageIndex;
}
```

Question 2:

I'm trying to use your DBTreeView component in a Delphi 3 application I am writing. It works fine until I try to enable the SmartRecordLoad option. When I enable it and try to open my table, I receive an "Invalid Variant Type Conversion" error.

Answer:

You have not assigned the Root Value. DBTreeView tries to load root records. It uses the locate YourDataSet.Locate method('YOURKEYFIELD', ROOTVALUE, []); You have to set RootValue to the 'root' records value of the key field. These values must be equal.

Question 3:

I try to use smart record loading. I indexed my Table and set `trSmartRecordLoad` to `True`. When I start my application the `DBTreeView` doesn't display any nodes. The nodes are displayed only when I press the 'Next' button within `DBNavigator`. Is this a feature of the smart record loading?

Answer:

No, You forgot to assign the `RootValue` property. Set the `PARENT` value of your 'root' records to this property.

Smart record loading works in the following way:

When activating a dataset, it loads records which have the `PARENT` value equal to `RootValue` (Locate the first record that the parent Value equals to `RootValue` and then fetch and load records while the value of the `PARENT` field doesn't change. That is why your dataset has to be sorted by the `PARENT` field). Then `DBTreeView` loads records when necessary – on node expansion and when `DBTreeView` has to retrieve the full structure of the loaded tree node.

In your particular case `DBTreeView` does the following:

1. Recognizes that you set smart record loading.
2. Checks `RootValue` and decides that the `PARENT` value of your 'root' records is `NULL`.
3. Tries to locate the record with the `PARENT` value `NULL`. When there is no such record, `DBTreeView` does not load any records at all.
4. When you press the 'Next' button within `DBNavigator`, `DBTreeView` checks `trCanDBNavigate` and since it is `True` (by definition it is `True`) loads the current record (to set it as a selected tree node) and also recursively loads all its parent records.

Question 4:

We want to use a different field for a Node's text, according to the node level, how can we do this ?

Answer:

1. You have to set `ReadOnly` to `True` to disable text editing in `DBTreeView`.
2. Write the code on the `OnDisplayItemText` event handler to assign your Text to a particular `TreeNode`:

```

procedure TForm1.DBTreeViewSetDisplayItemText(Sender: TObject;
  var DisplayText: string);
Var
  tr : TTreeNode;
begin
  // Get TreeNode
  tr := DBTreeView.DBTreeNodes.GetTreeNode(TreeQueryID.Value);
  if tr <> Nil then begin
    if tr.Level mod 2 = 1 then
      DisplayText := Fiedl1.Text
    else DisplayText := ' Fiedl2.Text;
  end;
end;

//-----
void __fastcall TForm1::DBTreeViewSetDisplayItemText(TObject *Sender,
  AnsiString &DisplayText)
{
  // Get TreeNode
  TTreeNode * tr =

```

```

DBTreeView->DBTreeNodes>GetTreeNode(TreeQueryID->Value);
if(tr != NULL)
{
    if((tr->Level && 1) == 1)
        DisplayText = Fiedl1->Text;
    else DisplayText = Fiedl2->Text;
}
}
//-----

```

3. Add DBEdit to a Form, bind it with your DataSet

4. Write the code on the OnChange event handler to change the DataField of DBEdit to the required FieldName.

```

procedure TForm1.DBTreeViewChange(Sender: TObject; Node: TTreeNode);
begin
    if(DBTreeView.Selected <> Nil) then begin
        if (DBTreeView.Selected.Level mod 2 = 1) then
            DBEdit.DataField := 'field1'
        else DBEdit.DataField := 'field2';
        end;
    end;
//-----
void __fastcall TForm1::DBTreeViewChange(TObject *Sender, TTreeNode *Node)
{
    if(DBTreeView->Selected != NULL){
        if((DBTreeView->Selected->Level && 1) == 1)
            DBEdit->DataField = "field1";
        else DBEdit->DataField = "field2";
    }
}

```

Question 5:

We would like to use specific fields for the state and image index values, depending on the level.

Answer:

You have to write the code on the AddNewItem event handler to change the ImageIndex property of a Tree Node.

```

procedure TForm1.DBTreeViewAddNewItem(Sender: TObject;
    var DBTreeNode: TdxDBTreeNode);
begin
    if(DBTreeNode <> Nil) then begin
        DBTreeNode.ImageIndex := DBTreeNode.Level;
        DBTreeNode.StateIndex := DBTreeNode.Level;
    end;
end;

//-----
void __fastcall TForm1::DBTreeViewAddNewItem(TObject *Sender,
    TdxDBTreeNode *&DBTreeNode)
{
    if(DBTreeNode != NULL)
    {
        DBTreeNode->ImageIndex = DBTreeNode->Level;
    }
}

```



```

        DBTreeNode->StateIndex = DBTreeNode->Level;
    }
}

```

Question 6:

I am trying to insert a child into my tree. My table structure is below, it is recursive by nature. I use the dbTreeView component.

Table structure

ID - auto
Parent - integer
Name - integer lookup

Lookup for Name

ID - auto
Name- String
Category - Integer
Obsolete - logical (true,false)

What method should I use to insert the child since I really need to send the Name as an integer not as a string? The addChild method takes a string. I need to set an integer value.

Answer:

Solutions are as follows:

The First Option:

1. Create a lookup field in the dataset

The Second Option:

1. Set DBTreeView ReadOnly to True.
2. Create a one-to-many relationship between your DataSets. DataSet1 is the second table (lookup table), DataSet2 is the first table (the self-referenced table) DataSet1 ->> DataSet2, BUT DataSet2 (self-reference) is the master DataSet.
3. Write the code on the OnDisplayItemText event handler to assign an appropriate Text

```
procedure TForm1.DBTreeViewSetDisplayItemText(Sender: TObject;
```

```
    var DisplayText: string);
```

```
begin
```

```
    DisplayText := DataSet1.FindField('name').AsString
```

```
end;
```

```
//-----
```

```
void      __fastcall      TForm1::DBTreeViewSetDisplayItemText(TObject  
*Sender,AnsiString &DisplayText)
```

```
{
```

```
    DisplayText = DataSet1->FindField("name")->Text;
```

```
}
```

Question 7:

I use your DBTreeView with the SmartRecordLoad option. When I delete the root record or a record that has > 100 children, the operation takes a great amount of time. Can you decrease the time of such operation? (I use InterBase).

Answer:

No, DBTreeView can not decrease the time of delete operations when using SmartRecordLoad, but you can do it manually.

When you delete a tree node (or a record, it does not matter), DBTreeView checks whether SmartRecordLoad is active. If it is set, DBTreeView first loads all 'child' records of the deleted record recursively and then deletes them. It works fast with ISAM databases, and you are right that it takes a lot of time if you use C/S architecture.

But with C/S architecture you can easily decrease this time dramatically. Instead of using the standard DBTreeView delete operation you have to carry out such operations in the following way:

1. Create a trigger on Delete. For example (for Interbase):

```
REFTABLE.ID <--->> REFTABLE.PARENT
Create Trigger REFTABLE$DELETE For REFTABLE
After Delete
As
Begin
    Delete From REFTABLE Where PARENT = OLD.ID And ID <> OLD.ID;
End
```

Now when deleting a record all children will be deleted automatically.

2. Set trCanDBDelete to False and obtain the KeyDown event manually:

```
procedure TForm1.DBTreeViewKey1Down(Sender: TObject;
var Key: Word; Shift: TShiftState);
Var
    Query : TQuery;
begin
    // Is it delete key and has the user selected a tree node
    if(Key = VK_DELETE) And (DBTreeView1.Selected <> Nil) then
        begin
            // Create a temporary query
            Query := TQuery.Create(self);
            // Set the DataBaseName
            Query.DataBaseName := 'MyDataBase';
            // Assign the SQL property
            Query.SQL.Add('Delete From REFTABLE Where ID = ' +
                          MYDBTreeDataSet.FindField('id').AsString);
            // Delete the record and our trigger
            //has to delete its children recursively
            Query.ExecSQL;
            // Free the temporary query
            Query.Free;
            //Refresh the DataSet
            MYDBTreeDataSet.Close;
            MYDBTreeDataSet.Open;
        end;
    end;

//-----
void __fastcall TForm1::DBTreeViewKeyDown(TObject *Sender,
WORD &Key, TShiftState Shift)
{
    // Is it delete key and has the user selected a tree node
    if((Key == VK_DELETE) && (DBTreeView1->Selected != NULL))
    {
        // Create a temporary query
        TQuery * Query = new TQuery(this);
```

```
// Set the DatabaseName
Query->DataBaseName = "MyDataBase";
// Assign the SQL property
Query->SQL->Add("Delete From REFTABLE Where ID = " +
               MYDBTreeDataSet->FindField("id")->Text);
// Delete the record and our trigger
// has to delete its children recursively
Query->ExecSQL();
// Free the temporary query
Query->Free();
// Refresh our DataSet
MYDBTreeDataSet->Close();
MYDBTreeDataSet->Open();
}
}
```

Question 8:

The following code does not work when automatic Drag-Drop is active, but does work when it is inactive:

```
procedure TForm1.DBTreeViewMouseUp(Sender: TObject;
Button: TMouseButton; Shift: TShiftState; X, Y: Integer);
begin
    if ssShift in Shift then begin
        {do something}
    end;
end;
```

Answer:

This is not a problem created by ExpressDBTreeView. Try to perform this operation using any other TWinControl in Delphi or CPB; for instance a ListBox control. The result will be the same. Instead of using "if ssShift in Shift then begin" condition you have to write the following code:

```
"if GetKeyState(VK_SHIFT) < 0 then begin".
procedure TForm1.DBTreeViewMouseUp(Sender: TObject;
Button: TMouseButton; Shift: TShiftState; X, Y: Integer);
begin
    if GetKeyState(VK_SHIFT) < 0 then begin
        {do something}
    end;
end;
```

Index

A	
Alignment.....	31
Alignment.....	36
AssignField.....	33
B	
BkColor.....	17
C	
CanSelectParents	22
ChildLoaded	19
CloseUp.....	27
Color	18
Count.....	15
D	
DataField.....	32, 37
DataSource	6, 32, 37, 45
DBSelected.....	6
DBTreeNode.....	6
DisplayField	7, 33
DividedChar	21
DoCustomDraw	40
DoDragDropIsCopy	40
DoDragDropTreeNode.....	40
DragSourceTreeNode.....	39
DropDown.....	28
DropDownRows	22
DropDownWidth	22
F	
FAQ	46
Field	31, 37
FontName.....	18
FontStyle	18
G	
GetDBTreeNode.....	16
GetImageIndexField	10
GetKeyField	10
GetKeyFieldValue	15
GetListField.....	10
GetParentField.....	10
GetParentValue.....	16
GetStateIndexField	10
GetTreeNode.....	16
GetTreeNodeByText.....	30
GotoKeyFieldValue	11
I	
ImageIndexField.....	7, 33
IsCustomDraw	11, 18, 40
Items.....	14, 30
K	
KeyField	7, 34, 45
KeyFieldValue	19
L	
LevelCount	45
ListField.....	7, 34
ListSource	34
LoadChildren	20
M	
MaxKeyFieldValue	14
MaxLevelCount	45
MoveTreeNodeStructure	41
O	
OnAddNewItem	11, 35
OnCloseUp.....	28
OnCreateNewKeyValue	12
OnCustomDraw.....	28, 41
OnDragDropIsCopy.....	42
OnDragDropTreeNode	42
OnDropDown.....	29
OnEndDragTreeNode	43
OnGetImageIndex	29
OnGetSelectedIndex.....	29
OnSetDisplayItemText	13, 36
Options	9, 34, 38
P	
ParentField.....	8, 34, 46
ParentFieldValue.....	20
R	
ReadOnly.....	32, 38
RefreshItems	11
RootValue	8, 35, 45
S	
Selected.....	22
SelectedIndex	39
Self-Referenced Structure	5
SeparatedSt	8
ShowNodeHint.....	39
Smart Record Loading.....	5
StateIndexField	8, 35
T	
TCustomdxLookupTreeView.....	32
TCustomdxTreeView	38
TCustomdxTreeViewEdit	29

TCustomdxVTreeViewEdit	20	TreeViewFont.....	23
TdxDBLookupTreeView	37	TreeViewHint.....	24
TdxDBTreeNode	16	TreeViewImages.....	24
TdxDBTreeNodes.....	14	TreeViewIndent.....	24
TdxDBTreePrintData.....	44	TreeViewPopupMenu	25
TdxDBTreeView	5	TreeViewReadOnly	25
TdxDBTreeViewEdit.....	31	TreeViewShowButtons	25
TdxTreeView	44	TreeViewShowHint	26
TdxTreeViewEdit.....	30	TreeViewShowLines.....	26
Text.....	22	TreeViewShowRoot.....	26
TextStyle.....	21	TreeViewSortType.....	27
TreeViewColor.....	23	TreeViewStateImages	27
TreeViewCursor	23		